b) Using a self-contained token format such as JWTs

c) Using a cloud-native database that can scale up to hold all the tokens

d) Using the `HmacTokenStore` in addition to the `DatabaseTokenStore`

e) Reusing an existing token when the same capability has already been issued

4. Which is the main reason why HATEOAS is an important design principle when using capability URIs? Pick one answer.

a) HATEOAS is a core part of REST

b) Capability URIs are hard to remember

c) Clients can’t be trusted to make their own URIs

d) Roy Fielding, the inventor of REST, says that it’s important

e) A client can’t make their own capability URIs and so can only access other resources through links

9.2.4 Hardening capability URIs

In section 9.2.1, I mentioned that putting the token in the URI path or query parameters is less than ideal because these can leak in audit logs, `Referer` headers, and through your browser history. A partial solution to this is to instead put the token in a part of the URI that is not usually sent to the server or included in `Referer` headers. The original